Project idea

Yk W

May 2026

1 Methodology: Prefix-Level Monitoring via Tree-Derived Scores

We consider an agent trajectory as a sequence of prefixes. After step k, the agent has observed the task, produced a sequence of actions, received environment feedback, and reached a current state. We denote this information by

$$P_k = (h_0, a_1, o_1, \ldots, a_k, o_k),$$

where h_0 is the original task description, a_t is the agent action at step t, and o_t is the corresponding observation. The goal is to monitor whether the current prefix is still likely to lead to eventual task success.

1.1 Prefix-Level Solvability Score

Instead of treating each step independently, we define a prefix-level solvability score

$$Y_k := \Pr(\text{eventual success} \mid P_k).$$

Here, Y_k measures how healthy the current trajectory prefix is. A high value of Y_k means that the current prefix still has a high probability of leading to a successful final solution, while a low value indicates hidden degradation even if the current step has not yet failed.

In practice, Y_k is estimated from a recursive binary rollout tree. At selected fork points, the current sandbox state is copied and two child branches are rolled out independently. Each leaf is evaluated by the final task outcome:

$$y(\ell) = \begin{cases} 1, & \text{if leaf } \ell \text{ resolves the task,} \\ 0, & \text{otherwise.} \end{cases}$$

For an internal node v, its empirical solvability score is defined as the average success rate of leaves in its subtree:

$$\hat{Y}(v) = \frac{1}{|\mathcal{L}(v)|} \sum_{\ell \in \mathcal{L}(v)} y(\ell),$$

where $\mathcal{L}(v)$ is the set of leaves descending from node v. Thus, each tree node receives a label representing the downstream success probability of the prefix rooted at that node.

1.2 Node Representation

For each node v, we construct a feature vector

$$x_v = \phi(P_v),$$

where P_v is the prefix corresponding to node v. The feature vector contains lightweight monitoring signals such as depth, token usage, context length, command type, return code, observation tag, repeated-command score, repeated-file score, and failure streak.

The key point is that we do not assign an independent parameter to every node. Instead, we learn a transferable scoring function

$$s(v) = x_v^\top \theta.$$

This allows the monitor to generalize from previously observed trees to new trees, because a new node can be scored through its prefix features.

1.3 Weighted Pairwise Bradley–Terry Learning

To learn the scoring function, we convert the rollout tree into weighted pairwise comparisons between sibling nodes. For two sibling nodes u and v sharing the same parent, their tree-derived solvability scores $\hat{Y}(u)$ and $\hat{Y}(v)$ estimate the downstream success probability of the corresponding prefixes. If

$$\hat{Y}(u) > \hat{Y}(v),$$

then node u is treated as a healthier continuation than node v, written as

$$u \succ v.$$

Unlike an unweighted comparison, we also use the magnitude of the solvability gap. A comparison such as $\hat{Y}(u) = 0.90$ and $\hat{Y}(v) = 0.10$ should provide stronger evidence than a comparison such as $\hat{Y}(u) = 0.51$ and $\hat{Y}(v) = 0.49$. Therefore, each sibling comparison is assigned a weight

$$w_{uv} = \left|\hat{Y}(u) - \hat{Y}(v)\right|.$$

Optionally, comparisons with very small gaps can be removed by requiring

$$\left|\hat{Y}(u) - \hat{Y}(v)\right| \geq \epsilon,$$

where $\epsilon > 0$ is a small threshold used to discard nearly tied and potentially noisy comparisons.

We model the probability that u is healthier than v using a Bradley–Terry form:

$$\Pr(u \succ v) = \frac{\exp(s(u))}{\exp(s(u)) + \exp(s(v))}.$$

Using the linear prefix score

$$s(v) = x_v^\top \theta,$$

this becomes

$$\Pr(u \succ v) = \sigma\left((x_u - x_v)^\top \theta\right),$$

where

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

is the logistic function.

Let $z_{uv} \in \{0, 1\}$ denote the pairwise preference label:

$$z_{uv} = \begin{cases} 1, & \text{if } \hat{Y}(u) > \hat{Y}(v), \\ 0, & \text{if } \hat{Y}(u) < \hat{Y}(v). \end{cases}$$

The parameter θ is estimated by maximizing the weighted pairwise log-likelihood

$$\ell(\theta) = \sum_{(u,v)\in\mathcal{D}} w_{uv} \left[z_{uv} \log \sigma\left((x_u - x_v)^\top \theta\right) + (1 - z_{uv}) \log\left(1 - \sigma\left((x_u - x_v)^\top \theta\right)\right)\right],$$

where $\mathcal{D}$ is the set of constructed sibling comparisons after optional filtering. The weight w_{uv} makes comparisons with large downstream solvability gaps contribute more strongly to the objective, while near-tie comparisons have little influence.

1.4 Monitoring Rule

Once $\hat{\theta}$ is learned, any new node v can be assigned a monitoring score

$$\hat{s}(v) = x_v^\top \hat{\theta}.$$

The score can be converted to a calibrated success probability by fitting a calibration map g, for example logistic calibration:

$$\widehat{Y}_{\text{cal}}(v) = g(\hat{s}(v)).$$

The monitor then tracks the score along a trajectory. A prefix is flagged as degraded if its estimated solvability falls below a threshold:

$$\widehat{Y}_{\mathrm{cal}}(v) < \tau,$$

or if the score drops sharply relative to its parent:

$$\Delta(v) = \hat{s}(v) - \hat{s}(\mathrm{parent}(v)) < -\delta.$$

The first rule detects absolutely low-quality prefixes. The second rule detects sudden degradation after a local decision.

1.5 Uncertainty-Aware Monitoring

To avoid overreacting to noisy estimates, we also estimate uncertainty in the score. Let

$$I(\hat{\theta})$$

be the observed Fisher information matrix from the weighted pairwise likelihood. Then

$$\widehat{\mathrm{Var}}(\hat{\theta}) \approx I(\hat{\theta})^{-1}.$$

For a node v, the variance of its score is approximated by

$$\widehat{\mathrm{Var}}(\hat{s}(v)) = x_v^\top I(\hat{\theta})^{-1} x_v.$$

Thus the standard error is

$$SE(\hat{s}(v)) = \sqrt{x_v^\top I(\hat{\theta})^{-1} x_v}.$$

A conservative lower confidence bound can be defined as

$$LCB(v) = \hat{s}(v) - c \cdot SE(\hat{s}(v)),$$

where $c = 1.96$ gives an approximate 95% confidence bound. The monitoring rule becomes

$$LCB(v) < \tau_s.$$

This rule flags a node only when its score is sufficiently low after accounting for uncertainty.